



دانشگاه شریف

درس ساختار کامپیوتر و میکروپروسسور

برگه‌ی سؤالات امتحان

تحلیل و طراحی معماری پردازنده‌ی PARS-16

مدت زمان آزمون	نوع آزمون	نمره‌ی کل
180 دقیقه (3 ساعت)	کتاب‌باز، تمام توضیحات لازم در همین جزوه آمده	150 نمره

راهنمای پاسخ‌دهی

- بخش «آشنایی با پردازنده‌ی PARS-16» در همین جزوه (صفحه‌ی بعد) در طول آزمون همراه شماست و نیازی به حفظ کردن جدول‌ها نیست؛ هر زمان لازم شد به آن برگردید.
- در صورت ابهام در صورت سؤال، مفروضات خود را صریح بنویسید و ادامه دهید.

جمع	از 150

سؤال	1	2	3	4	5	6	7
از	15	10	20	20	10	35	40
نمره							

آشنایی با پردازنده‌ی PARS-16

این بخش، یک معرفی کامل و خودکفا از پردازنده‌ای به نام PARS-16 است. لازم نیست از قبل چیزی درباره‌ی این پردازنده بدانید. تمرکز این آزمون روی تحلیل و طراحی معماری است، نه رمزگذاری صرف دستور؛ به همین دلیل، علاوه بر مجموعه دستور، مسیر داده و سیگنال‌های کنترلی هم در همین جا معرفی می‌شوند. پیشنهاد می‌شود پیش از رفتن سراغ سؤالات، این بخش را کامل بخوانید.

ایده‌ی کلی

PARS-16 یک پردازنده‌ی آموزشی ساده است: تمام دستورهایش دقیقاً 16 بیت طول دارند، تعداد دستورهایش کم و مشخص است، و فرمت دستورها بسیار منظم است. یک ویژگی خاص هم دارد: به جای آنکه فقط دستورهای پرش «شرطی» باشند (مثل بیشتر پردازنده‌های رایج)، هر دستوری می‌تواند شرطی اجرا شود.

مشخصات کلی

مقدار	مشخصه
16 بیت	عرض کلمه (داده و دستور)
8 عدد: R0 تا R7 (R0 همیشه صفر است)	رجیسترهای عمومی
Harvard (حافظه‌ی دستور و داده کاملاً جدا)	معماری حافظه
Word-addressed (هر آدرس یک کلمه‌ی 16 بیتی است، نه یک بایت)	آدرس دهی
16 دستور (تمام فضای opcode پر می‌شود)	تعداد کد عملیات

نوشتن روی R0 همیشه بی‌اثر است و خواندن از آن همیشه 0 می‌دهد. این کار یک مدار ترکیبی کوچک است که ورودی فعال‌سازی نوشتن رجیستر فایل را وقتی آدرس مقصد برابر صفر باشد، خاموش نگه می‌دارد.

فرمت‌های دستور

در تمام فرمت‌ها، دو فیلد در موقعیت ثابتی قرار دارند: CC (کد شرط، دو بیت، بیت 15 تا 14) و OPCODE (کد عملیات، چهار بیت، بیت 13 تا 10).

MSB (bit 15)			LSB (bit 0)		
CC	OPCODE	Rd	Rs1	Rs2	S

فرمت R؛ عملیات ثابت به ثابت

فرمت R برای دستورهای محاسباتی/منطقی ثابت محور است (مثل ADD): Rd مقصد، Rs1 و Rs2 دو منبع، و S بیت فعال‌سازیِ فلگ هستند (توضیح در ادامه). این تنها فرمتی است که بیت S دارد.

MSB (bit 15)			LSB (bit 0)	
CC	OPCODE	Rd/Rs2	Rs1	imm4

فرمت I؛ عملیات با مقدار فوری کوچک

فرمت I برای دستورهایی با یک مقدار فوری 4 بیتی علامت‌دار به کار می‌رود (مثل SW/LW). در دستور SW، فیلدی که معمولاً «مقصد» خوانده می‌شود در واقع رجیستر منبع داده‌ای است که باید در حافظه نوشته شود.

MSB (bit 15)			LSB (bit 0)	
CC	OPCODE	Rd	imm7	

فرمت U؛ بارگذاری مقدار فوری بزرگ‌تر

فرمت U برای دستورهایی مثل LUI/MOVI است که یک مقدار فوری 7 بیتی مستقیماً در رجیستر مقصد می‌گذارند. توجه کنید این فرمت اصلاً فیلد Rs1 ندارد.

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	offset10	

فرمت B/J؛ پرش نسبی

فرمت B/J برای دستورهای پرش (B^۱ و JAL^۲) است. چون خود فیلد CC شرط پرش را می‌سازد، نیازی به فیلد رجیستر نیست و کل فضای باقی‌مانده (10 بیت) به یک آفست نسبی اختصاص می‌یابد. این آفست نسبت به آدرس دستور بعدی (یعنی PC+1) حساب می‌شود.

جدول کامل دستورات

Opcode	Mnemonic	فرمت	S	عملکرد
0000	ADD	R	بله	$Rd = Rs1 + Rs2$
0001	SUB	R	بله	$Rd = Rs1 - Rs2$
0010	AND	R	بله	$Rd = Rs1 \& Rs2$
0011	OR	R	بله	$Rd = Rs1 Rs2$
0100	XOR	R	بله	$Rd = Rs1 \wedge Rs2$
0101	SHL	R	بله	شیفت چپ Rs1 به اندازه‌ی Rs2[3:0]
0110	SHR	R	بله	شیفت راست Rs1 به اندازه‌ی Rs2[3:0]
0111	ADDI	I	خیر	$Rd = Rs1 + s\text{-}ext(imm4)$
1000	LW	I	خیر	$Rd = MEM[Rs1 + s\text{-}ext(imm4)]$
1001	SW	I	خیر	$MEM[Rs1 + s\text{-}ext(imm4)] = Rs2$
1010	MOVI	U	خیر	$Rd = s\text{-}ext(imm7)$
1011	LUI	U	خیر	$Rd[15:9] = imm7, Rd[8:0] = 0$
1100	B	B	خیر	اگر CC برقرار: $PC = PC+1+s\text{-}ext(offset10)$
1101	JAL	J	خیر	اگر CC برقرار: $PC=PC+1+s\text{-}ext(offset10)$ ، $R7=PC+1$
1110	JR	JR	خیر	اگر CC برقرار: $PC = Rs1$
1111	SYS	SYS	خیر	HALT :func=0001؛ NOP :func=0000

¹ Branch

² Jump And Link

شبه‌دستور **CMP**: $CMP\ Rs1, Rs2$ فقط یک نام خواناتر برای $SUB\ R0, Rs1, Rs2, S$ است یعنی تفریق را انجام می‌دهد، فلگ‌ها را به‌روز می‌کند، اما چون مقصد $R0$ است نتیجه دور ریخته می‌شود. نحو نشانی دهی حافظه: $LW\ Rd, imm(Rbase)$ یعنی $Rd = MEM[Rbase + imm]$ به همین ترتیب برای SW .

آفست پرش: در دستورهای JAL/B ، مقدار $offset10$ نسبت به آدرس دستور بعدی (یعنی $PC+1$) حساب می‌شود: $offset$ برابر است با آدرس مقصد منهای (آدرس خود دستور پرش $+1$).

رجیستر فلگ و اجرای شرطی سراسری

ویژگی متمایزکننده‌ی PARS-16 این است: هر دستور، بدون استثنا، دو بیت شرط اجرا (CC) دارد. پیش از اجرای واقعی هر دستور، کنترلر بررسی می‌کند آیا شرط آن دستور با وضعیت فعلی یک «رجیستر فلگ» برقرار است یا نه. اگر برقرار نباشد، دستور عملاً به یک NOP تبدیل می‌شود: هیچ رجیستری نوشته نمی‌شود، هیچ حافظه‌ای تغییر نمی‌کند، فلگ‌ها دست‌نخورده می‌مانند و PC فقط یک واحد افزایش پیدا می‌کند (حتی اگر دستور خودش یک پرش بوده باشد).

رجیستر فلگ از چهار فلگ تشکیل شده:

فلگ	شرح
(Zero) Z	یک می‌شود اگر نتیجه‌ی آخرین عملیات فلگ‌گذار دقیقاً صفر باشد
(Negative) N	برابر با بیت علامت (بیت 15م) نتیجه است
(Carry) C	نقل رقم خروجی جمع‌کننده (برای SUB/ADD)
(oVerflow) V	سرریز علامت‌دار (فرمول زیر)

فرمول سرریز برای $ADD\ (Rd = Rs1 + Rs2)$: $V=1$ اگر علامت $Rs1$ و $Rs2$ یکسان باشد ولی علامت Rd با آن‌ها متفاوت باشد. برای $SUB\ (Rd = Rs1 - Rs2)$: $V=1$ اگر علامت $Rs1$ و $Rs2$ متفاوت باشد و علامت Rd با علامت $Rs1$ متفاوت باشد. برای دستورهای منطقی/شیفت، همیشه $C=0$ و $V=0$ است.

بیت S: فقط هفت دستور فرمت R این بیت را دارند. اگر $S=1$ و شرط CC همان دستور برقرار باشد، رجیستر فلگ با نتایج تازه به روز می شود؛ در غیر این صورت فلگ ها دست نخورده می مانند. هیچ دستور دیگری اصلاً بیت S در قالب خود ندارد و به هیچ وجه فلگ عوض نمی کند.

کد شرط دوبیتی:

کاربرد معمول	شرط اجرا	نام	CC
پیش فرض بیشتر دستورها	همیشه اجرا شود	AL (Always)	00
برابری، پایان حلقه ی شمارشی	$Z=1$	EQ (Equal)	01
مقایسه ی علامت دار (کوچک تر بودن)	$N=1$	LT (Less Than)	10
تشخیص سرریز حسابی	$V=1$	VS (oVerflow Set)	11

با فقط 2 بیت (4 حالت که یکی «همیشه» است)، فقط سه شرط واقعی در دسترس است؛ شرط های معکوس مثل NE (نامساوی) به طور مستقیم وجود ندارند.

پرش بر اساس مقایسه: چون هیچ دستور پرشی اجازه ی فعال سازی فلگ ندارد، برای «پرش بر اساس یک مقایسه» به دو دستور پشت سر هم نیاز است: ابتدا یک دستور SUB با $S=1$ که مقصدش R0 است (نتیجه ی تفریق دور ریخته می شود، اما فلگ ها درست تنظیم می شوند)، و بلافاصله بعد از آن یک دستور B با کد شرط مناسب که همان فلگ های تازه را می خواند.

```
CMP R1, R2 ; flags are updated based on R1-R2
B.EQ Loop ; if R1==R2, jump to Loop
```

شبیه سازی شرط های معکوس: برای رسیدن به شرطی مثل NE که مستقیماً وجود ندارد، از الگویی دو-دستوری استفاده می شود: نخست یک پرش شرطی با $CC=EQ$ به سمت نقطه ای بلافاصله بعد از دستور دوم می رود (یعنی اگر برابر بودند، از پرش بعدی «رد می شویم»); سپس یک پرش بدون قید (AL) به مقصد واقعی می رود. اگر مقادیر برابر نبودند، پرش اول (چون EQ برقرار نیست) عملاً NOP می شود و اجرا مستقیماً به پرش بدون قید می رسد.

```

CMP    R1, R2

B.EQ   Skip    ; if equal, skip over the unconditional jump
B.AL   Target  ; otherwise (not equal), jump unconditionally
Skip:

```

یک مثالِ کامل

برنامه‌ی زیر سه عدد را با هم جمع کرده و نتیجه را در `MEM[0]` ذخیره می‌کند:

```

MOVI R1, 5
MOVI R2, 7
MOVI R3, 3
ADD  R4, R1, R2
ADD  R4, R4, R3
SW   R4, 0(R0)
HALT

```

برای آنکه با روندِ کدگذاریِ دستی آشنا شوید، دستورِ اول (`MOVI R1, 5`) را قدم‌به‌قدم کدگذاری می‌کنیم.

قدم ۱ - تعیینِ فرمت: `MOVI` طبقِ جدولِ بخشِ قبل از فرمتِ `U` است.

قدم ۲ - تعیینِ مقدارِ هر فیلد:

مقدار	فیلد
00 (بدون قید)	CC
1010 (کد MOVI)	OPCODE
001 (یعنی R1)	Rd
0000101 (یعنی عدد 5 در 7 بیت)	imm7

قدم ۳ - کنار هم گذاشتن فیلدها: فیلدها را دقیقاً به همان ترتیب فرمت U (یعنی CC، سپس OPCODE، سپس Rd، سپس imm7) پشت سر هم می‌چسبانیم تا یک رشته‌ی 16 بیتی به دست آید:

00 1010 001 0000101

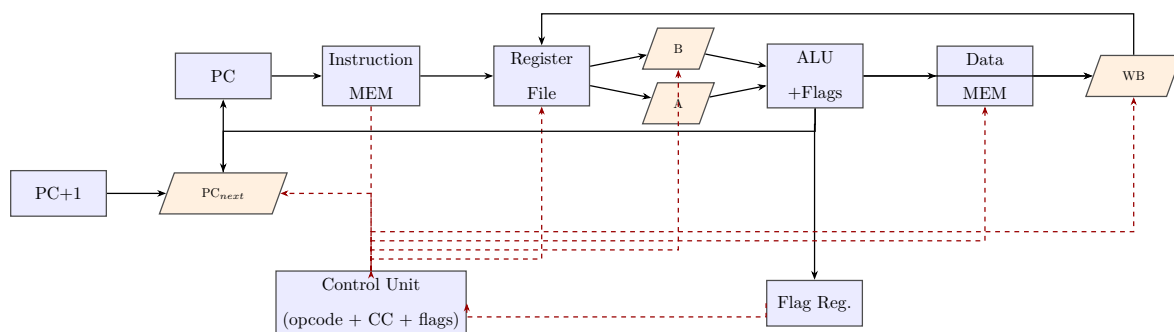
قدم ۴ - تبدیل به هگز: همین رشته را در گروه‌های 4 تایی بازنویسی می‌کنیم:

$$0010\ 1000\ 1000\ 0101_2 = 0x2885$$

بقیه‌ی دستورهای برنامه هم دقیقاً با همین چهار قدم (تعیین فرمت، تعیین فیلدها، چسباندن فیلدها، تبدیل به هگز) کدگذاری می‌شوند. پس از اجرای کامل برنامه (7 سیکل کلاک)، مقدار MEM[0] برابر با 15 خواهد بود.

مسیر داده و سیگنال‌های کنترلی

شکل زیر مسیر داده‌ی تک‌سیکلی PARS-16 را نشان می‌دهد (نمودار مفهومی است، نه سیم‌کشی دقیق). واحد کنترل با خواندن OPCODE دستور و ارزیابی CC در برابر رجیستر فلگ، مجموعه‌ای از سیگنال‌های کنترلی تولید می‌کند که مسیر حرکت داده در بقیه‌ی مدار را تعیین می‌کنند.



این هفت سیگنال کنترلی، دقیقاً معلوم می‌کنند هر جعبه/مالتی پلکسر چه کاری انجام دهد:

سیگنال	توضیح
RegWrite	فعال‌سازی نوشتن در پورت نوشتن رجیستر فایل
RegDst	آدرس مقصد نوشتن: از فیلد Rd دستور، یا همیشه R7 (فقط برای JAL)
MemRead	فعال‌سازی خواندن از Data MEM (برای LW)
MemWrite	فعال‌سازی نوشتن در Data MEM (برای SW)
ALUSrc	انتخاب‌گر مالتی پلکسر B: ۰ = مقدار Rs2، ۱ = مقدار فوری علامت‌گسترده شده
WBSEL	انتخاب‌گر مالتی پلکسر WB: ۰ = خروجی ALU، ۱ = داده‌ی خوانده شده از حافظه
PCSrc	انتخاب‌گر مالتی پلکسر PC _{next} : توالی عادی (PC+1)، یا هدف پرش

یک نکته‌ی حیاتی: مقدار RegWrite و MemWrite (و انتخاب غیر «توالی عادی» در PCSrc) همیشه با بیت cond_met (نتیجه‌ی ارزیابی CC دستور در برابر فلگ‌ها) AND می‌شوند؛ یعنی حتی اگر واحد کنترل بر اساس OPCODE این سیگنال‌ها را «۱» تولید کند، اگر شرط دستور برقرار نباشد، مقدار نهایی

صفر خواهد بود و هیچ تغییر حالتی رخ نمی‌دهد. همین منطق برای فعال‌سازی نوشتن رجیستر فلگ هم برقرار است: رجیستر فلگ فقط وقتی به‌روز می‌شود که هم بیت S دستور یک باشد، هم شرط آن دستور برقرار باشد.

یک مثال کامل: ردیابی سیگنال برای یک دستور

فرض کنید دستور ADD R3, R1, R2 (بدون شرط، بدون فعال‌سازی فلگ، یعنی $S=0$) با مقادیر $R1=5$ و $R2=9$ اجرا می‌شود.

سیگنال/کمیت	مقدار
RegWrite	1 (این دستور از گروه R-type است)
RegDst	R3 (از فیلد Rd)
MemWrite / MemRead	0 (این عملیات به حافظه دسترسی ندارد)
ALU	Rs2 یعنی مقدار R2 (چون $\text{ALUSrc}=0$ برای فرمت R) ورودی B به ALU
خروجی ALU	$5 + 9 = 14$
منبع داده‌ی نوشته‌شونده	ALU (چون $\text{WBSe1}=0$)
FlagWrite	0 (چون $S=0$ است، صرف‌نظر از شرط)

یعنی در پایان این سیکل، $R3=14$ می‌شود و رجیستر فلگ دست‌نخورده می‌ماند.

سؤال ۱

(۱۵ نمره)

این سؤال سه بخش مستقل دارد؛ برای هر بخش، استدلال فنی پاسخ‌تان اهمیت دارد، نه فقط نتیجه.

الف) (۵ نمره) چرا در پردازنده‌ی PARS-16 برای حافظه‌ی دستور و حافظه‌ی داده از دو حافظه‌ی جداگانه (معماری هاروارد^۱) استفاده شده، نه یک حافظه‌ی مشترک (معماری فون‌نویمان^۲)؟ این انتخاب دقیقاً چه کمکی به سادگی مسیر داده‌ی تک‌سیکل^۳ می‌کند؟

ب) (۵ نمره) در این پردازنده، فقط دستورهای فرمت R یک بیت فعال‌سازی فلگ (به نام S) دارند و دستورهایی مانند پرش اصلاً چنین بیتی در قالب خود ندارند. اگر دستور پرش هم اجازه‌ی تغییر فلگ‌ها را داشت، چه مشکلی می‌توانست در نوشتن برنامه‌های شرطی ایجاد کند؟

ج) (۵ نمره) گفته شد نوشتن روی R0 همیشه بی‌اثر است. این رفتار توسط یک مدار ترکیبی کوچک تضمین می‌شود. توضیح دهید این مدار احتمالاً کجا قرار دارد و چه کاری انجام می‌دهد (به کدام سیگنال کنترل و به کدام بخش رجیستر فایل مرتبط است)؛ اگر این مدار حذف شود، دقیقاً چه اتفاقی برای دستور ADD R0, R1, R2 می‌افتد؟

سؤال ۲

(۱۰ نمره)

برای هرکدام از دو دستور زیر، با استفاده از مسیر داده و جدول سیگنال‌های کنترلی بخش «آشنایی با پردازنده»، جدول مقابل آن را کامل کنید. صرفاً نوشتن مقادیر کافی نیست؛ برای هر ردیف یک جمله‌ی

¹ Harvard Architecture

² von Neumann Architecture

³ Single-Cycle Datapath

کوتاه هم بنویسید که چرا آن مقدار درست است.

الف) (۵ نمره) دستور SUB R2, R4, R1, S بدون قید، یعنی همیشه اجرا می شود) با مقادیر $R4=10$ و $R1=3$.

سیگنال/کمیت	مقدار و دلیل
RegWrite	
RegDst	
ورودی های A و B به ALU	
خروجی ALU	
FlagWrite و مقادیر V, N, Z	

ب) (۵ نمره) دستور LW R5, 4(R3) با مقدار $R3=20$ و $MEM[24]=99$.

سیگنال/کمیت	مقدار و دلیل
MemWrite و MemRead	
ALUSrc و خروجی ALU	
WBSe1	
مقدار نهایی R5	

سؤال ۳

(۲۰ نمره)

فرض کنید می‌خواهیم دستور جدیدی به نام *PARS-16 LWI (Load Word & Increment)* اضافه کنیم، با این رفتار:

$$Rs1 = Rs1 + 1 \text{ سپس } , Rd = MEM[Rs1] \Rightarrow \text{LWI } Rd, Rs1$$

یعنی یک کلمه از آدرس داخل $Rs1$ می‌خواند و همزمان خود $Rs1$ را یکی زیاد می‌کند (برای پیمایش آرایه‌ها بدون نیاز به یک دستور جمع جداگانه در هر تکرار).

الف) (۵ نمره) این دستور به کدام فیلدها نیاز دارد؟ کدام یک از فرمت‌های موجود ($B/J, U, I, R$) را می‌توان، حتی با چشم‌پوشی از فیلدهای اضافه/بلااستفاده، برای آن استفاده کرد؟ توضیح دهید.

ب) (۵ نمره) همان‌طور که در جدول دستورها دیدید، هر ۱۶ مقدار فیلد *OPCODE* از قبل استفاده شده‌اند. با توجه به اینکه دستور *SYS* خودش از یک فیلد فرعی *func* برای تفکیک *NOP* از *HALT* استفاده می‌کند، بدون افزایش عرض کلی دستور (همچنان ۱۶ بیت)، یک راه برای جا دادن *LWI* در فضای دستور پیشنهاد دهید.

ج) (۱۰ نمره) این بخش، اصل چالش طراحی است. با نگاه به مسیر داده‌ی بخش «آشنایی با پردازنده»: مسیر داده‌ی فعلی برای اجرای *LWI* در یک سیکل دقیقاً چه کمبودی دارد؟ (توضیح دهید مسیر داده فعلی چه محدودیتی برای اجرای این دستور دارد).

راه‌حلی برای مشکل بخش (ج) پیشنهاد دهید (مثلاً افزودن سخت‌افزار جدید، یا تقسیم اجرا به بیش از یک سیکل) و توضیح دهید این راه‌حل دقیقاً چه تغییری در مسیر داده یا واحد کنترل ایجاد می‌کند.

سؤال ۴

(۲۰ نمره)

الف) (۱۰ نمره) طراحان این پردازنده عرضِ کدِ شرط را 2 بیت انتخاب کرده‌اند (که فقط سه شرط واقعی، غیر از AL، در اختیار می‌گذارد). فرض کنید می‌خواستیم علاوه بر EQ، LT و VS، شرط NE (نامساوی) را هم به‌طور مستقیم پشتیبانی کنیم.

• چند بیت برای کدِ شرط لازم می‌شد؟

• این تغییر، با فرض ثابت ماندنِ عرضِ کلیِ دستور (16 بیت) و عرضِ فیلدِ OP CODE (4 بیت)، چه اثری روی عرضِ فیلدِ مقدارِ فوری در فرمتِ I (imm4) می‌گذاشت؟ عرضِ جدید را دقیقاً محاسبه کنید.

ب) (۱۰ نمره) حالا فرض کنید به‌جای گسترشِ کدِ شرط، می‌خواهیم بیتِ S (فعال‌سازیِ فلگ) را به تمام فرمت‌ها اضافه کنیم، نه فقط فرمتِ R.

• این تغییر روی فرمتِ I (که در حال حاضر: $CC(2)+OPCODE(4)+Rd(3)+Rs1(3)+imm4(4)=16$) بیت است) چه اثری می‌گذارد؟ عرضِ جدیدِ imm4 را با نشان دادنِ معادله‌ی جمعِ بیت‌ها محاسبه کنید.

• آیا به‌نظر شما این مبادله (بیتِ S برای همه در برابرِ عرضِ کوچک‌ترِ مقدارِ فوری) ارزشش را دارد؟ با یک دلیلِ فنی پاسخ دهید.

سؤال ۵

(۱۰ نمره)

PARS-16 به هر دستور اجازه‌ی اجرای شرطی می‌دهد (اجرای پیش‌بینی‌شده^۱)، برخلافِ معماری‌هایی مانند MIPS/RISC-V که فقط دستور پرش می‌تواند شرطی باشد.

¹ Predicated Execution

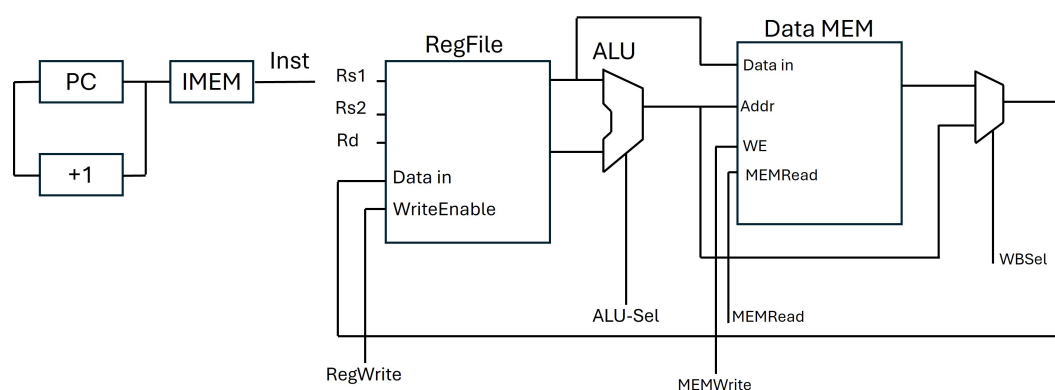
الف) (۵ نمره) یک برنامه‌ی کوتاه با دستورهای PARS-16 بنویسید که مقدار $R3$ را برابرِ بزرگ‌ترِ $R1$ و $R2$ قرار دهد ($R3 = \max(R1, R2)$)، بدون استفاده از هیچ دستور پرشی. سپس در یک یا دو جمله توضیح دهید چرا یک معماری فقط-پرش-شرطی برای همین کار معمولاً به دستورهای بیشتری نیاز دارد.

ب) (۵ نمره) اجرای شرطی سراسری (این‌که هر دستور کد شرط داشته باشد) رایگان نیست. یک هزینه یا محدودیت این طراحی، در مقایسه با معماری فقط-پرش-شرطی، را از نظر سخت‌افزاری یا برنامه‌نویسی توضیح دهید.

سؤال ۶

(۳۵ نمره)

مدار شکل زیر یک مدل ساده اولیه از مسیر داده پردازنده را نمایش می‌دهد. این مدل تنها توانایی اجرای دستورات به فرم $R_k = \text{ALU}(R_i, R_j)$ و دستورات LW و SW را دارد.



شکل ۱: شکل مدار با فرمت‌های R

الف) (۲۵ نمره) مدار را به گونه‌ای تغییر دهید که توانایی اجرای دستورات پرش، دستورات به فرم $R_k = \text{ALU}(R_i, \text{SignExtend}(\text{Imm}))$ و MOVI را داشته باشد.

ب) (۱۰ نمره) جدول زیر سیگنال‌های کنترلی موجود در مدار بالا را نمایش می‌دهد. اگر سیگنال کنترلی

جدیدی در قسمت الف به مدار اضافه کرده‌اید آن را نیز به این جدول اضافه کنید و سپس جدول را برای دستورات گفته شده کامل کنید. اگر هر یک از سیگنال‌های کنترلی برابر با بخشی از دستور هستند می‌توانید از فرمت $Inst[?:??:?]$ و یا نام آن قسمت از دستور (s, Rd, etc.) استفاده کنید.

RegWrite	MemRead	MemWrite	ALU-Sel	WBSel	--	--
(JAL)						
(MOVD)						
(ADD)						
(ADDI)						

برای مشخص کردن ALUSel از جدول زیر استفاده شود:

Operation	ALUSel
ADD	001
SUB	010
AND	011
OR	100
XOR	101

(۴۰ نمره)

سؤال ۷

می‌خواهیم به PARS-16 قابلیت پشته^۱ اضافه کنیم. رجیستر R6 اشاره‌گر پشته (SP) است و پشته رو به آدرس‌های کوچک‌تر رشد می‌کند. برای مثال، دستور PUSH Rs این‌گونه تعریف می‌شود:

$$MEM[R6] = R_s, R6 = R6 - 1 \Rightarrow \text{PUSH } R_s$$

¹ Stack

پشته (Stack) یک ساختار داده‌ای LIFO (آخرین ورودی، اولین خروجی) است که در حافظه اصلی (Data MEM) برای مدیریت فراخوانی توابع، ذخیره‌سازی متغیرهای محلی و نگهداری وضعیت پردازنده استفاده می‌شود.

ثبات اشاره‌گر (Stack Pointer - SP): ثباتی است که همیشه آدرس آخرین عنصر وارد شده به پشته (بالای پشته) را نگه می‌دارد که در این سوال از R6 استفاده می‌شود.

عملکرد: از دستورات PUSH Ri و PUSHI imm10 برای قرار دادن داده جدید روی پشته و از دستور POP Ri برای خواندن و برداشتن داده از روی پشته استفاده می‌شود. دقت کنید که برای برداشتن داده از روی پشته تنها خواندن داده و جابه‌جایی ثبات اشاره‌گر کافی است و نیازی به «پاک کردن» یا صفر کردن خانه حافظه نیست.

جهت رشد: در معماری PARS-16، پشته از آدرس‌های حافظه بالا به سمت آدرس‌های پایین (نزولی) رشد می‌کند.

شبه دستور: دستورات اسمبلی که مستقیماً به یک خط کد ماشین تبدیل نمی‌شود بلکه اسمبلر ابتدا آن را به چند خط دستور ترجمه می‌کند و سپس هر یک از آن خطوط دستور به یک خط کد ماشین تبدیل می‌شود. مثال: شبه‌اسمبلی PUSH R5 به اسمبلی زیر ترجمه می‌شود:

```
SW R5 0(R6)
```

```
ADDI R6 R6 -1
```

الف) (۵ نمره) ترجمه اسمبلی کدهای شبه‌اسمبلی POP R4 و PUSHI 40 را بنویسید.

ب) (۲۰ نمره) مدار سوال قبل را به گونه‌ای تغییر دهید که توانایی اجرای دستورات PUSH و PUSHI را به طور مستقیم و بدون نیاز به شبه‌دستور داشته باشد. (با توجه به پر بودن فضای Opcode فرض کنید دستورات LUI و SYS حذف شده‌اند تا جا برای این دو دستور باز شود. البته این موضوع فقط کنترلر را تغییر می‌دهد نه مسیر داده‌ای که از شما خواسته شده است)

ج) (۱۰ نمره) جدول زیر را برای مداری که طراحی کرده‌اید کامل کنید (مطابق سوال قبل)

RegWrite	MemRead	MemWrite	ALU-Sel	WBSel	--	--
<i>(PUSH)</i>						
<i>(PUSHH)</i>						

د) (۵ نمره) توضیح دهید پیاده‌سازیِ POP در سخت‌افزار، چرا و به چه تغییر در رجیستر فایل نیاز دارد؟